Vision-Based Model Predictive Control for Steering of a Nonholonomic Mobile Robot

This project report is submitted in partial fulfillment of the requirements of the coursework of ENPM667: Control of Robotic Systems

by

Khuzema Habib and Sounderya Varagur Venugopal
(UID: 121288776, 121272423)



James Clark School of Engineering

University of Maryland, College Park, MD

November 2024

Abstract

This paper introduces a novel visual servo-based Model Predictive Control (MPC) framework for steering a nonholonomic wheeled mobile robot (WMR) moving in a polar coordinate frame towards a desired target. The proposed method integrates both kinematic and dynamic control levels, where the kinematic controller generates desired velocity commands, and the dynamic controller computes the torques required to achieve these commands. The system leverages a Kinect sensor to provide visual feedback, enabling position-based servoing by extracting target features and determining the robot's pose relative to the target. The control design employs quadratic programming (QP) to optimize performance while adhering to constraints such as actuator saturation and velocity limits. A primal-dual neural network (PDNN) is utilized for solving QP problems in real time, effectively reducing computational load and enabling operation on resource-constrained hardware. Theoretical analysis guarantees stability and global convergence to optimal solutions, while experimental validation demonstrates the method's effectiveness and superiority over conventional approaches. By transforming the robot's kinematic model into polar coordinates, the proposed controller addresses challenges like nonholonomic constraints and singularities at the goal position, ensuring smooth and robust trajectory tracking. This work highlights the integration of visual servoing, MPC, and neural network optimization for efficient and reliable control of nonholonomic mobile robots in constrained environments. Simulations have been carried out on a TurtleBot model using ROS, Gazebo and Python to replicate the results concluded by the paper.

Keywords: Model Predictive Control(MPC), neurodynamics, Nonholonomic Mobile Robots (NMRs), Quadratic Programming (QP), visual servo steering, Neural networks

Contents

Abstract i

Contents ii

List of Figures iv

Abbreviations v

1 Introduction 1

2 Nonholonomic Mobile Robots Model 5
- 2.1 Kinematic Model . . . 6
- 2.2 Dynamics Model . . . 7
- 2.3 Camera Model . . . 10
- 2.4 Robot-Camera-Target Model . . . 11

3 Model Predictive Control Design 14
- 3.1 Model Predictive Control Scheme . . . 14
- 3.2 Kinematic Controller . . . 16
- 3.3 Dynamic Controller . . . 21

4 Neural Dynamics-Based Online Optimization 26

5 Stability Analysis 30

6 Experimental Results 34
- 6.1 Experimental Setup . . . 34
- 6.2 Control Implementation . . . 35
  - 6.2.1 MPC Controller . . . 35
  - 6.2.2 PD Controller . . . 36
  - 6.2.3 Primal Dual Neural Network . . . 36
- 6.3 Experimental Scenario . . . 38
- 6.4 Performance Evaluation . . . 38
- 6.5 Visualization . . . 40
- 6.6 Optimization Framework . . . 41

7 Conclusion **42**

Bibliography **45**

List of Figures

2.1 Caption 5
2.2 Block diagram of visual servo steering control system 12

4.1 Block diagram of primal-dual neural dynamic system 29

6.1 MPC Controller Trajectory 38
6.2 PD Controller Trajectory 39
6.3 PD Controller Velocities 39
6.4 MPC Controller Linear Velocity 39
6.5 MPC Controller Angular Velocity 40
6.6 PD Controller Gazebo Visualization 40
6.7 MPC Controller Gazebo Visualization 40

Abbreviations

MPC	**M**odel **P**redictive **C**ontrol
WMR	**W**heeled **M**obile **R**obot
NMRs	**N**on **H**olonomic **R**obots
RH	**R**eceding **H**orizon
QP	**Q**uadratic **P**rogramming
PDNN	**P**rimal **D**ual **Neural Network**
LVI	**L**inear **V**ariational **I**nequalities

Chapter 1

Introduction

The use of visual feedback-based motion control for nonholonomic mobile robots (NMRs) has gained significant attention in robotics research over recent decades. Most studies in this area focus on position-based visual servoing, which involves reconstructing the target pose relative to the robot. This reconstruction typically leads to Cartesian motion control problems, such as vision-based tracking and stabilization.

A brief review of vision-based tracking of NMR is provided as below. In [1], a single visual-based tracking controller was proposed for both static and moving objects in a 2-D plane. The visual feedback was used to control the robot pose for tracking a moving object of interest. In [2], a two-level position-based visual servoing scheme was proposed to ensure global convergence and field-of-view (FOV) constraint for a mobile robot equipped with a pan camera. In [3], in order to control an NMR to track a desired trajectory without directly measuring its position, a vision-based tracking combining an adaptive estimator was proposed, which estimates the global position of the mobile robot online using natural visual features measured by a vision system.

For the vision-based stabilization of NMR, a number of typical research works are reviewed as below. In [4], a hybrid visual-based stabilization strategy was presented on a motion-estimation technique, and it can be applied in both planar and nonplanar scenes and requires no matrix estimation or decomposition. In [5], a robust stabilization controller based on a two-phase technique was presented, which can stabilize the mobile robot to the

origin in the absence of the depth information and precise visual parameters. The work mentioned above only takes the kinematics of a mobile robot into account. As a matter of fact, there have been only a few studies in which both kinematics and dynamics of the mobile robot are considered for the visual servo steering. In [6], an adaptive sliding-mode dynamic stabilizing controller was proposed for an uncertain nonholonomic dynamic mobile robot when the camera was fixed in the ceiling. In [7], a dynamic feedback control law based on the backstepping technique was presented for a WMR to perform the position-based visual steering.

Most reported studies do not address the internal constraints such as actuator saturation and velocity limitation. Model Predictive Control, also know as receding horizon(RH) control can be used to incorporate these constraints into the control designs and handle them through optimization procedures. In [8], an RH controller was developed for tracking control of an NMR but the computation of the RH controller in real-time systems was not addressed. In [9], a model-predictive trajectory-tracking control applied to a mobile robot was presented, and in [10], the tracking-error kinematic model for mobile robots is linearized as well at the equilibrium point and transferred to an exact discrete-time form including the time-delay. However, both control laws were derived from a quadratic cost function using the linearized dynamics. In [11], a nonlinear model predictive formation controller for target perception was implemented to drive omnidirectional mobile robots converge toward a desired target. However, the above works [8]–[11] did not consider the visual servo control.

This paper considers a complete set of constraints, including nonholonomic and the internal constraints, to develop a visual servo steering control of an NMR, which is equipped with an on-board Kinect vision sensor. The objective is to drive the mobile robot from an arbitrary initial position to a given target position, using the visual data obtained from the Kinect depth camera. The design of the controller is divided into two parts. The first one is presented for the kinematic model and the other one is applied to the dynamic model. This paper utilizes the feature of MPC together with the neural network optimization for the visual servo control design of the mobile robot, which is subject to a number of internal kinematics and dynamics constraints. Through optimizing an objective function

repeatedly and utilizing the dynamics model, the developed visual-based MPC generates the most suitable control input for dynamics systems given the system constraints.

MPC control has been well applied in trajectory following and exploration of WMR. In [12], the path-tracking of an autonomous mobile robot has been proposed using learning-based nonlinear MPC (NMPC), based on a priori vehicle model and a learned disturbance model. Vision-based autonomous exploration with WMR studied in [13] only relies on measurements from visual sensors and wheel encoders. The resulting state estimate is combined with visual information to build an environment map, which is then exploited by an MPC scheme to define trajectories favoring unexplored locations. In [14], the learning-based MPC uses a linear model with bounds on its uncertainty to construct invariant sets that provide deterministic guarantees on robustness and safety. In [15], an RH MPC technique was proposed to address path following and obstacle avoidance through geometric singularities and discontinuities. Research in [9] uses a linearized tracking-error dynamics to predict future system behavior, and uses a quadratic cost function of tracking error and the control effort to derive a control law. Studies in the above mentioned work focus on the trajectory following, while in this paper, steering such a vehicle to approach a given target by the vision feedback has been investigated. The difficulty of this nonholonomic motion planning and control problem for WMR lies in the fact that due to the limits imposed by the Brocketts result, a goal position and orientation are not reachable asymptotically by means of smooth and time invariant feedback control laws. In addition, the traditional MPC method used in these papers would consume much online computational resource. While in this paper, we propose a novel visual-based NMPC enhanced by neural network optimization and visual servoing. Such a controller allows online operation in real time even when computing power is severely limited. As a matter of fact, the neural network optimization approaches have been developed as a promising approach for addressing the problem caused by the heavy online computational load. Neural networks for constrained optimization problems have recently been investigated for real-time applications [16], [17]. In [18], the coordination of kinematically redundant robots was investigated using a dual neural-network method, where both the physical limits of both joint torques and the generalized forces applied on the object were considered. In [19], a three-layer neural network was applied to linear

MPC to compute exact optimal solution. In [20], a neural network was proposed for solving linear variational inequalities (LVIs) and quadratic optimization using projection techniques.

Therefore, to significantly reduce the computational load, this paper proposes a novel visual-based NMPC enhanced by neural network optimization and visual servoing. Such a controller allows online operation in real time even in the cases where computing power is severely limited. This is particularly useful to common mobile robots on which the equipped computer is usually not very powerful due to cost performance requirement. The use of the neural network for optimization allows a significant level of parallelization in hardware that is capable of supporting it. It is worth to mention that the other computational efficient MPC methods, such as explicit MPC (eMPC), can also be applied for online control of mobile robot. However, an eMPC solution is limited to piecewise linear models, such that the nonlinear systems need to be linearized for the procedure to be applicable [28], while the primal-dual neural network (PDNN)-based MPC proposed in this paper requires no *a priori* knowledge on linearization. Consider the highly nonlinear model and a large number of constraints on the mobile robot model, control design based on the original nonlinear model is much more straightforward than that based on linearized model.

The main contributions of this paper can be summarized as follows:

- By incorporating MPC into visual-based stabilization in a novel manner, the robust and optimal steering of an NMR has been achieved, in the presence of both kinematic and dynamic constraints.
- An enhanced MPC algorithm with computational efficiency has been proposed using a neural network enabled QP method.
- Both theoretical analysis and experimental tests have been performed to validate the effectiveness of the proposed scheme, and to demonstrate its superiority over conventional approach.

Chapter 2

Nonholonomic Mobile Robots Model

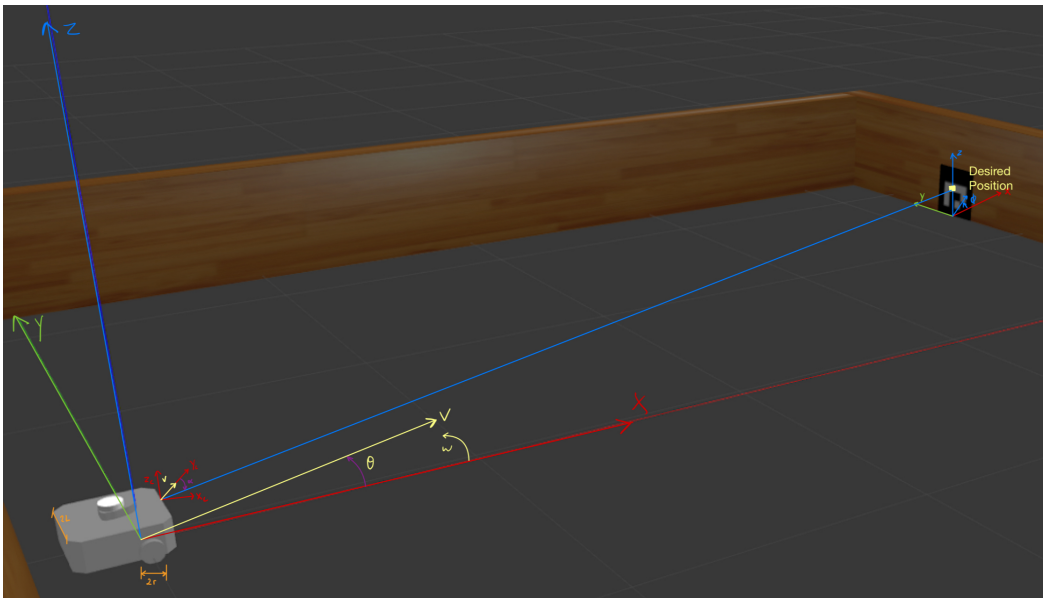


FIGURE 2.1: Caption

The coordinates of the WMR are as shown in the above figure. OXYZ is the global coordinate frame attached on the ground, $OX_cY_cZ_c$ is a coordinate frame attached to the RGBD camera, and $OX_bY_bZ_b$ is the coordinate frame attached to the goal. The orientation of Y_c and the linear velocity v are the same. It is assumed that the mobile robot is in the absence of lateral slip so that the translational velocity is only in the direction of heading.

2.1 Kinematic Model

The kinematic model of a mobile robot describes its motion in terms of its position and orientation with respect to a global coordinate frame. For a WMR, the assumption of nonholonomic constraints - no lateral slipping or skidding, plays a key role in defining the equations of motion. Let the position of the robot in teh global frame OXYZ be represented by (x,y), and its orientation by θ. The state vector can thus be defined as:

$$q = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}$$

. The velocities governing the motion of the robot are:

- **Linear velocity**(v): Speed in the direction in which the robot is facing.
- **Angular velocity**(ω): Rotational speed around the vertical axis.

The relationship between the velocities and the rate of change of the robot's position and orientation is expressed by the kinematic model:

$$\dot{x} = v\cos\theta, \quad \dot{y} = v\sin\theta, \quad \dot{\theta} = \omega \tag{2.1}$$

The same can be written in a compact form as $\dot{q} = J(q)u$, where

$$J(q) = \begin{bmatrix} \cos\theta & 0 \\ \sin\theta & 0 \\ 0 & 1 \end{bmatrix}, u = \begin{bmatrix} v \\ \omega \end{bmatrix} \tag{2.2}$$

$J(q)$ is the Jacobian matrix that captures the geometric relationship between the global frame and the local frame of the robot. It transforms the control inputs into the corresponding rate of change in the robot's pose.

2.2 Dynamics Model

The dynamic model includes the physical characteristics of the robot, such as mass, inertia, friction, and external forces. This level of modeling is essential for accurately predicting the robot's behavior under the influence of forces and torques. The model is derived using Newton-Euler equations, which describe the motion of a rigid body under the influence of forces and torques. Assuming that the center of mass and the coordinate center of the WMR are the same, the dynamics of the mobile robot can be described by the three-generalized coordinates and inputs $\tau \in \mathbb{R}^2$ as

$$M(q)\ddot{q} + V_m(q,\dot{q})\dot{q} + G(q) + F(\dot{q}) + \tau_d = B(q)\tau - A^T(q)\lambda \tag{2.3}$$

where

- m: Mass of the robot
- I: Moment of inertia about the robot's vertical axis
- $2L$: Distance between the two driven wheels
- r: Radius of the wheel
- $M(q) \in \mathbb{R}^{3\times 3}$: The mass matrix, representing the inertial properties of the roboot, given by

$$M(q) = \begin{bmatrix} m & 0 & 0 \\ 0 & m & 0 \\ 0 & 0 & I \end{bmatrix}$$

- $V_m(q,\dot{q})$: Coriolis and centripetal forces arising rom the robot's motion. For simplicity, this is often neglected for planar robots
- $G(q)$: Gravitational forces, which can be ignored in a planar system

$$V_m(q,\dot{q}) = 0, G(q) = 0$$

- $F(\dot{q}) \in \mathbb{R}^3$: Frictional forces and torques action on the robot due to ground friction, opposing its linear and angular velocities

- τ: Applied torques from the motors, given by:

$$\tau = \begin{bmatrix} \tau_l \\ \tau_r \end{bmatrix}$$

  where τ_r and τ_l are the torque control inputs generated by the right and left dc motor, respectively

- $\tau_d \in \mathbb{R}^3$: Bounded unknown disturbances

- $B(q) \in \mathbb{R}^{3\times 2}$: The input transformation matrix, mapping the motor torques τ to the robot's dynamics given by

$$B(q) = \frac{1}{r}\begin{bmatrix} \cos\theta & \cos\theta \\ \sin\theta & \sin\theta \\ L & -L \end{bmatrix}$$

- $\lambda \in \mathbb{R}$: Lagrange multipliers representing the reaction forces required to enforce the robot's nonholonomic constraints, given by

$$\lambda = -m(x\cos\theta + \dot{y}\sin\theta)\dot{\theta}$$

- $A^T(q)$: The transpose of the constraint matrix(Nonholonomic constraints are velocity-level constraints that limit the robot's motion in certain directions, such as lateral slipping). This matrix maps constraint forces into the robot's generalized coordinates and is given by

$$A^T(q) = \begin{bmatrix} -\sin\theta \\ \cos\theta \\ 0 \end{bmatrix}$$

  Together, $A^T(q)\lambda$ ensures that the robot's motion adheres to its nonholonomic constraints.

The matrix $A(q)$ encodes constraints that limit motion, making them **kinematic** constraints. In essence, the matrix represents the directions in which the robot's motion is constrained. The Jacobian matrix, $J(q)$, describes the directions in which the robot

can move in space. To ensure **kinematic consistency** of the model, the motion allowed by the Jacobian and the motion limited by the constraints should not interfere with each other. This make the the velocity vectors orthogonal and therefore, the matrix product is 0.

$$J^T(q)A^T(q) = 0$$

Using the above equation and premultiplying (2.3) by $J^T(q)$ on both sides, we get

$$\bar{M}(q)\dot{u} + \bar{V}_m(q,\dot{q})u + \bar{F}(\dot{q}) + \bar{\tau}_d = \bar{B}(q)\tau \tag{2.4}$$

where

$$\bar{M} = J^T(q)M(q)J(q) \in \mathbb{R}^{2\times 2}, \quad \bar{V}_m = J^T(q)\big[M\dot{J}(q) + V_m J(q)\big] \in \mathbb{R}^{2\times 2},$$

$$\bar{F} = J^T(q)F \in \mathbb{R}^2, \quad \bar{\tau}_d = J^T(q)\tau_d, \quad \bar{B} = J^T(q)B,$$

$$u = \begin{bmatrix} v \\ \omega \end{bmatrix} \in \mathbb{R}^2.$$

. This gives

$$\bar{M}(q) = \begin{bmatrix} m & 0 \\ 0 & I \end{bmatrix}, \quad \bar{B} = \frac{1}{r}\begin{bmatrix} 1 & 1 \\ L & -L \end{bmatrix}.$$

By regarding the ground friction and the disturbance torque as the modeling uncertainties and disturbances, then the dynamic equation (2.4) of the simple model of the NMR is given by:

$$\bar{M}\dot{u} + \bar{F} + \bar{\tau}_d = \bar{B}\tau \tag{2.5}$$

For a two wheeled robot, we can further simplify this equation by defining $\tau_1 = (\tau_l + \tau_r)/r$ and $\tau_2 = L(\tau_l - \tau_r)/r$. The dynamic equation (2.5) becomes:

$$\tilde{M}\dot{u} + \tilde{F} + \tilde{\tau}_d = \bar{\tau} \tag{2.6}$$

where $\bar{\tau} = [\tau_1 \ \tau_2]^T$ denotes linear and angular torques, respectively, which act on the body of the mobile robot.

2.3 Camera Model

The camera model is used to show the projective relationship between the target point in the 3-D Euclidean space and its corresponding point in the 2-D image plane. The visual camera mounted on the mobile robot is the Microsoft Kinect sensor, with a RGB color camera in the middle of the device, as well as an infrared (IR) transmitter and an IR CMOS camera on both sides. An RGB camera delivers three basic color components (red, green, and blue) on three different wires. The Kinect is also a 3-D depth sensor and uses light coding technology, that is, it uses an active IR light phase difference of the back and forth to measure the depth. The Kinect projects some random lattice through IR receiver lens. After the reflection of coarse objects of the measurement space, it can use an ordinary CMOS sensor to capture the lattice. In simple terms, when the depth of the scene changes, the captured lattice will change as well. By combining the depth information with RGB image data, the full 3D position of each point in the environment can be reconstructed.

The 3D position of any point in the environment is represented in the **camera frame**, denoted as $[x_c, y_c, z_c]^T$. The **world frame** is a fixed global coordinate system, while the **camera frame** is a moving local coordinate system attached to the camera.

The **perspective model** relies on the principle of perspective geometry, where objects appear smaller as their distance from the camera increases. The Microsoft Kinect sensor uses this model to map 3D points in the camera frame $[x_c, y_c, z_c]^T$ to 2D points on the image plan $[s, p]^T$ based on the geometry of similar triangles. The projection equations are:

$$|s - s_0| = f_k \frac{x_c}{y_c}, \quad |p - p_0| = f_k \frac{z_c}{y_c} \tag{2.7}$$

where,

- $[s_0, p_0]^T$: Coordinates of the principal point, representing the center of the image plane in pixel
- $[x_c, y_c, z_c]^T$: 3-D coordinate of the desired position with respect to the Kinect frame
- y_c: Depth axis

- $[s, p]^T$: Corresponding image pixel coordinate with respect to the boxes frame $X_b Y_b Z b$
- f_k: Focal length of the Kinect sensor

  The physical interpretation of this is that as the the depth z_c increases, the projected 2D coordinates (s, p) move closer to the principal point, reflecting how objects appear smaller when farther away.

Feature-point matching between the RGB image and the depth image is able to spatially correlate those feature points between two different images. The **RGB image** provides a detailed color representation of the environment but lacks depth information whereas, it is the vice-versa for the **depth image**. This spatial mapping helps feature points to obtain their true depth values based on the RGB camera's coordinate system. Meanwhile, the depth camera measures 3-D coordinates of those feature points in the IR receiver camera's coordinate system. Visual appearance of the object observed is obtained by mapping the color image onto the depth map and IR image. We use our calibration parameters and the rectification technique [21] to remove image distortions and align the color and depth images. Therefore, we can obtain the position information from the visual feedback to realize the robot position-based servoing. Position-Based Servoing (PBS) is a visual servoing technique where the control system uses the 3D position and orientation (pose) of a target, relative to the robot, as the feedback input to guide the robot's motion. This is achieved by reconstructing the target's position in the robot's coordinate frame (or a global coordinate frame) and using this information to compute control actions.

As shown in Fig 2.1, the position-based control is presented, the goal features are extracted from the image, and can be used in conjunction with a geometric model of the goal to determine the pose of the target with respect to the Kinect.

2.4 Robot-Camera-Target Model

Most kinematic models of mobile robots describe the robot's position and orientation using a Cartesian coordinate system (x, y, θ). Brockett's theorem [22] states that mooth

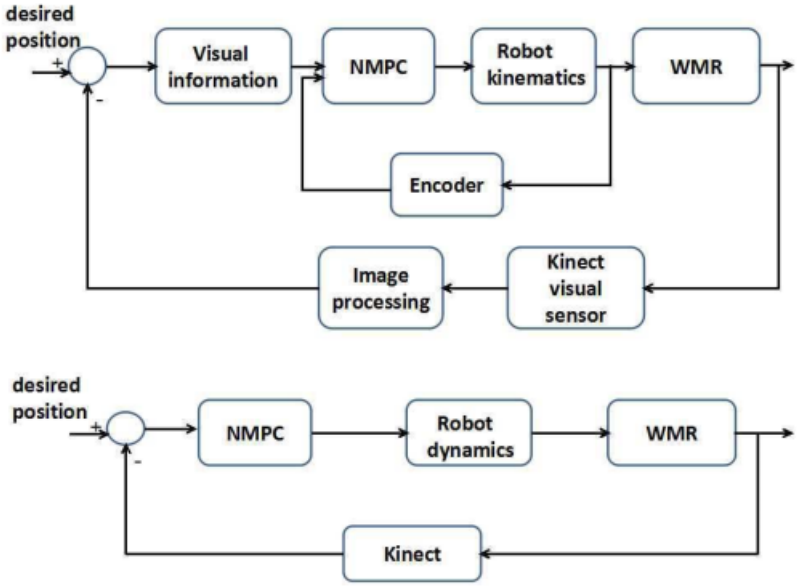


FIGURE 2.2: Block diagram of visual servo steering control system

and time-invariant feedback control laws cannot asymptotically stabilize a nonholonomic system (like a wheeled mobile robot) at a point in Cartesian coordinates. The robot's feasible motion is constrained, and the state-space in Cartesian form does not accommodate this inherent structure effectively. This limits the ability of control systems to smoothly and globally stabilize nonholonomic robots to a specific point and orientation in Cartesian space using time-invariant feedback.

If a different state-space representation is adopted, simpler approaches can be used, directly allowing smooth steering properties, for example, finding suitable strategies that allow the mobile robot to reach the goal. In particular, we want to determine the steering strategies that asymptotically drive the mobile robot toward a desired position and orientation. For this, the kinematics in (2.1) can be transformed from Cartesian coordinate (x_c, y_c, θ) in the Kinect frame into a polar state (e, ϕ, α) via:

$$\begin{aligned} e &= \sqrt{x_c^2 + y_c^2} \\ \phi &= \text{atan2}(-y_c, -x_c) \\ \alpha &= \phi - \theta \end{aligned} \tag{2.8}$$

where $x_c = e\cos\phi$ and $y_c = e\sin\phi$. The kinematic equation(2.1) can thus be transformed into polar coordinate

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{e} \\ \dot{\phi} \\ \dot{\alpha} \end{bmatrix} = \begin{bmatrix} -v\cos\alpha \\ \frac{v\sin\alpha}{e} \\ \frac{v\sin\alpha}{e} - \omega \end{bmatrix}. \tag{2.9}$$

If we choose the goal position as origin, since the goal frame is fixed, the velocity and angular velocity of the mobile robot in the goal frame are the same.

Chapter 3

Model Predictive Control Design

The MPC has become an increasingly popular control technique used in industrial applications with various constraints. The control problem is formulated as a cost function, then converted into a nonlinear constraint optimization problem. The behavior of the system is predicted based on the system model over a predictive horizon at each sampling time(A predictive horizon refers to a fixed time window into the future. For example, if the horizon is 2 seconds and the system operates at 10 updates per second, the prediction will look 20 steps ahead).Then, using the predicted information, a future sequence of control input is obtained by minimizing the objective cost function. Only the first control input of the optimal control sequence is used. The same procedure is repeated using the updated measurements and a shifted horizon at the next sampling time. In this section, a visual steering controller, which consists of the kinematic controller and dynamic controller, both of which are based on the MPC to obtain the optimal inputs, is designed to ensure that the mobile robot can be driven to the desired position.

3.1 Model Predictive Control Scheme

Consider a discrete nonlinear system in the general form as

$$\mathbf{x}(k+1) = \mathbf{g}(\mathbf{x}(k), u(k)) \tag{3.1}$$

subject to constraints

$$\mathbf{x}(k) \in \mathcal{X}, \quad k = 1, 2, \ldots, N \tag{3.2}$$

$$u(k) \in \mathcal{U}, \quad k = 1, 2, \ldots, N_u \tag{3.3}$$

where $\mathbf{x}(k) \in \mathbb{R}^3$ represents the state vector at time step k, $u(k) \in \mathbb{R}^2$ is the control input, and $g : \mathbb{R}^3 \times \mathbb{R}^2 \to \mathbb{R}^3$ is the system's nonlinear state-update function and is assumed to be continuous; $\mathcal{X} \in \mathbb{R}^3$ and $\mathcal{U} \in \mathbb{R}^2$ are compact(closed and bounded) sets that specifies the allowable range of states and permissible inputs respectively and contain the origin in their interiors; and N and N_u are prediction horizon $1 \leq N$ and control horizon $0 \leq N_u \leq N$, respectively.

The MPC is an iterative optimization technique: at each sampling time, measure or estimate the current state, and optimize a cost function to obtain an optimal input vector. Iteratively solving an optimization problem online makes the major difference between the MPC and standard control strategies, where the control laws are obtained in advance. Assuming that the control objective for system (3.1) is to steer the state to the origin, then the cost function for the MPC can be defined as

$$J(\mathbf{x}, u) = \sum_{j=k}^{k+N-1} L(\mathbf{x}(j), u(j)) + F(\mathbf{x}(k+N)) \tag{3.4}$$

where the stage cost $L(x, u)$ represents the immediate penalty associated with deviations in the system state x and control input u at each step. It is defined to satisfy the following properties:

1. **Zero at the Origin:** $L(0, 0) = 0$, meaning no cost is incurred when the system is at the desired state with no control effort.

2. **Non-Negativity:** $L(x, u) \geq c(|x, u|)^2$, where $c(\cdot)$ is a positive-definite function, ensuring that the cost is always positive except at the origin.

From a theoretical perspective, having an infinite prediction and control horizon ($N = N_u = \infty$) in the stage cost $L(x, u)$ is ideal, as it ensures stability by considering the

system's behavior over an unlimited future timeframe. However, such an approach is computationally impractical.

To address this limitation while maintaining stability and good closed-loop performance, the following modifications are incorporated into the finite-horizon MPC formulation:

1. **Terminal Cost:** A terminal cost $F(x(k+N))$ is added to approximate the infinite horizon effect by penalizing the system's state at the end of the prediction horizon.
2. **Terminal Constraint:** A terminal constraint $x(k+N) \in \Omega$ is introduced, where Ω is a region in the neighborhood of the origin. This ensures that the system reaches a desirable region within the prediction horizon, providing stability guarantees.

As a result, the following standard quadratic form of $L(\mathbf{x}, u)$ is most often used in practice:

$$L(\mathbf{x}, u) = \sum_{j=1}^{N} \mathbf{x}^T(k+j|k) Q \mathbf{x}(k+j|k) + \sum_{j=0}^{N_u-1} \Delta u^T(k+j|k) R \Delta u(k+j|k) \tag{3.5}$$

where $\mathbf{x}(k+j|k)$ denotes the predicted state in the future horizon, $\Delta u(k+j|k)$ denotes the input increment, where $\Delta u(k+j|k) = u(k+j|k) - u(k-1+j|k)$, and Q and R are appropriate weighting matrices. It was shown that by tuning N, N_u, Q, and R, the closed-loop stability in this case can also be achieved in practice [23] [24].

3.2 Kinematic Controller

The continuous-time system in (2.9) can be transformed into a discrete-time form using the zero-order (ZOH) discretization approach[25] with a sampling period as:

$$\mathbf{x}(k+1) = \mathbf{x}(k) + \dot{\mathbf{x}}(t)T. \tag{3.6}$$

Applying (2.9), we can rewrite (3.6) as

$$\mathbf{x}(k+1) = \begin{bmatrix} e(k+1) \\ \phi(k+1) \\ \alpha(k+1) \end{bmatrix} = \begin{bmatrix} e(k) - v(k)\cos\alpha(k)T \\ \phi(k) + \frac{v(k)\sin\alpha(k)T}{e(k)} \\ \alpha(k) + \frac{v(k)\sin\alpha(k)T}{e(k)} - \omega(k)T \end{bmatrix}. \tag{3.7}$$

To compute the optimal control input $u(k) = [v(k), \omega(k)]^T$ at the current time step k, the following finite-horizon optimal control problem is solved online:

$$u^* = \arg\min_u J(\mathbf{x}, u) \tag{3.8}$$

subject to the constraints

$$u_{\min} \leq u(k) \leq u_{\max} \tag{3.9}$$

$$\Delta u_{\min} \leq \Delta u(k) \leq \Delta u_{\max} \tag{3.10}$$

$$\mathbf{x}_{\min} \leq \mathbf{x}(k) \leq \mathbf{x}_{\max} \tag{3.11}$$

where $\Delta u(k)$ represents the change in the control input between successive time steps and $\Delta u_{\min}$ and $\Delta u_{\max}$ are its bounds. $\mathbf{x}_min$ and $\mathbf{x}_max$ are lower and upper bounds of the state vector. The kinematics equation in (3.7) can be written in the form

$$\mathbf{x}(k+1) = f(\mathbf{x}(k)) + g(\mathbf{x}(k))u(k) \tag{3.12}$$

where $f(\cdot)$ and $g(\cdot)$ are the continuous nonlinear functions, $f(0) = 0$; $\mathbf{x} = [x_1\, x_2\, x_3]^T = [e\, \phi\, \alpha]^T \in \mathbb{R}^3$ is the state vector; $u = [u_1\, u_2]^T = [v\, \omega]^T \in \mathbb{R}^2$ is the input vector; and

$$f(\mathbf{x}) = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}, \quad g(\mathbf{x}) = T \begin{bmatrix} -\cos x_3 & 0 \\ \sin x_3 / x_1 & 0 \\ \sin x_3 / x_1 & -1 \end{bmatrix}.$$

To predict the state across a horizon of N steps, we iteratively expand the kinematics model. $\mathbf{x}(k+i \mid k)$ represents the predicted state at time $k+i$, given the state at time k. Instead of predicting just one step ahead, MPC predicts multiple future states over a time horizon which gives the concatenated system's variables over the prediction horizon:

- $\tilde{\mathbf{x}} = \Big[\mathbf{x}(k+1 \mid k), \ldots, \mathbf{x}(k+N \mid k)\Big]^T \in \mathbb{R}^{3N}$ is the predicted state vector.
- $\tilde{u}(k) = \Big[u(k \mid k), \ldots, u(k+N_u-1 \mid k)\Big]^T \in \mathbb{R}^{2N_u}$ is the sequence of control inputs.
- $\Delta\tilde{u}(k) = \Big[\Delta u(k \mid k), \ldots, \Delta u(k+N_u-1 \mid k)\Big]^T \in \mathbb{R}^{2N_u}$ is the sequence of control input rates.

$$\Delta u = u(k) - u(k-1) \tag{3.13}$$

Substituting this in (3.12) gives

$$\mathbf{x}(k+1) = f(\mathbf{x}(k)) + g(\mathbf{x}(k))u(k-1) + g(\mathbf{x}(k))\Delta u(k) \tag{3.14}$$

Similarly, at $k+2$, the state is:

$$\mathbf{x}(k+2) = f(\mathbf{x}(k+1)) + g(\mathbf{x}(k+1))u(k+1) \tag{3.15}$$

Substituting $u(k+1) = u(k) + \Delta u(k+1)$:

$$\mathbf{x}(k+2) = f(\mathbf{x}(k+1)) + g(\mathbf{x}(k+1))(u(k) + \Delta u(k+1)) \tag{3.16}$$

Breaking this down further shows how $\Delta u(k)$ and $\Delta u(k+1)$ cascade forward to influence $\mathbf{x}(k+2)$. Over the prediction horizon N, the control input changes $\Delta u(k), \Delta u(k+1), \ldots$ collectively influence all future states $\mathbf{x}(k+1), \mathbf{x}(k+2), \ldots$. This cumulative influence is captured by the matrix G, which encodes:

1. The system's dynamics at each step $g(\mathbf{x}(k))$.
2. How each $\Delta u(k)$ propagates through the states.

The predicted output can then be expressed in the following form:

$$\bar{\mathbf{x}}(k) = G\Delta\bar{u}(k) + \tilde{f} + \tilde{g} \tag{3.17}$$

where

$$G = \begin{bmatrix} g(\mathbf{x}(k|k-1)) & \cdots & 0 \\ g(\mathbf{x}(k+1|k-1)) & \cdots & 0 \\ \vdots & \ddots & \vdots \\ g(\mathbf{x}(k+N-1|k-1)) & \cdots & g(\mathbf{x}(k+N-1|k-1)) \end{bmatrix} \in \mathbb{R}^{3N \times 2N_u}$$

$$\tilde{f} = \begin{bmatrix} f(\mathbf{x}(k|k-1)) \\ f(\mathbf{x}(k+1|k-1)) \\ \vdots \\ f(\mathbf{x}(k+N-1|k-1)) \end{bmatrix} \in \mathbb{R}^{3N}$$

$$\tilde{g} = \begin{bmatrix} g(\mathbf{x}(k|k-1))u(k-1) \\ g(\mathbf{x}(k+1|k-1))u(k-1) \\ \vdots \\ g(\mathbf{x}(k+N-1|k-1))u(k-1) \end{bmatrix} \in \mathbb{R}^{3N}$$

G is the system dynamics matrix derived from $g(x)$, $\tilde{f}$ accounts for the contribution of the function $f(\mathbf{x})$ and $\tilde{g}$ represents the effect of past control actions. The original optimization problem (3.5) subject to constraints (3.9) - (3.11) becomes

$$\min \|\bar{\mathbf{x}}(k)\|_Q^2 + \|\Delta\bar{u}(k)\|_R^2 = \min \|G\Delta\bar{u}(k) + \tilde{f} + g\|_Q^2 + \|\Delta\bar{u}(k)\|_R^2 \tag{3.18}$$

subject to:

$$\Delta\bar{u}_{\min} \leq \Delta\bar{u}(k) \leq \Delta\bar{u}_{\max} \tag{3.19}$$

$$\bar{u}_{\min} \leq \bar{u}(k-1) \leq \bar{u}_{\max} \tag{3.20}$$

$$\bar{u}_{\min} \leq \bar{u}(k-1) + \Delta\bar{u}(k) \leq \bar{u}_{\max} \tag{3.21}$$

$$\bar{\mathbf{x}}_{\min} \leq \bar{f} + \bar{g} + G\Delta\bar{u}(k) \leq \bar{\mathbf{x}}_{\max} \tag{3.22}$$

where $\Delta\bar{u}_{\min}, \Delta\bar{u}_{\max}, \bar{u}_{\min}$, and $\bar{u}_{\max}$ are the lower and upper bounds of the input vectors; $\bar{\mathbf{x}}_{\min}$ and $\bar{\mathbf{x}}_{\max}$ are the lower and upper bounds of the state vectors. They are all constant vectors, and

$$\bar{I} = \begin{bmatrix} I & 0 & \cdots & 0 \\ I & I & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ I & I & \cdots & I \end{bmatrix} \in \mathbb{R}^{2N_u \times 2N_u}.$$

Equation (3.18) can be expanded as:

$$\|\tilde{x} + G\Delta\tilde{u} + \tilde{f} + \tilde{g}\|_Q^2 = (\tilde{x} + G\Delta\tilde{u} + \tilde{f} + \tilde{g})^T Q(\tilde{x} + G\Delta\tilde{u} + \tilde{f} + \tilde{g}).$$

Expanding this quadratic form:

$$\|\tilde{x} + G\Delta\tilde{u} + \tilde{f} + \tilde{g}\|_Q^2 = \tilde{x}^T Q\tilde{x} + 2(\tilde{f} + \tilde{g})^T QG\Delta\tilde{u} + (G\Delta\tilde{u} + \tilde{f} + \tilde{g})^T Q(G\Delta\tilde{u} + \tilde{f} + \tilde{g}).$$

The $\tilde{x}^T Q\tilde{x}$ term is independent of $\Delta\tilde{u}$ and does not affect the optimization, so it can be ignored. What remains is:

$$\|\tilde{x} + G\Delta\tilde{u} + \tilde{f} + \tilde{g}\|_Q^2 = 2(\tilde{f} + \tilde{g})^T QG\Delta\tilde{u} + (\Delta\tilde{u})^T G^T QG(\Delta\tilde{u}) + \text{constant}.$$

The control effort term $\|\Delta\bar{u}(k)\|_R^2$:

$$\|\Delta\bar{u}(k)\|_R^2 = (\Delta\bar{u})^T R(\Delta\bar{u}).$$

Equation (3.18) can now be written as a QP problem

$$\min \frac{1}{2}\Delta\bar{u}^T W\Delta\bar{u} + c^T\Delta\bar{u} \tag{3.23}$$

subject to

$$E\Delta\bar{u} \leq b \tag{3.24}$$

$$\Delta\bar{u}_{\min} \leq \Delta\bar{u} \leq \Delta\bar{u}_{\max}, \tag{3.25}$$

where the coefficients are

$$W = 2(G^T QG + R) \in \mathbb{R}^{2N_u \times 2N_u} \tag{3.26}$$

$$c = -2G^T Q(\tilde{g} + \tilde{f}) \in \mathbb{R}^{2N_u} \tag{3.27}$$

$$E = \begin{bmatrix} -\tilde{I}, & \tilde{I}, & -G, & G \end{bmatrix}^T \in \mathbb{R}^{(4N_u + 6N) \times 2N_u} \tag{3.28}$$

$$b = \begin{bmatrix} -\bar{u}_{\min} + \bar{u}(k-1) \\ \bar{u}_{\max} - \bar{u}(k-1) \\ -\bar{\mathbf{x}}_{\min} + \tilde{f} + \tilde{g} \\ \bar{\mathbf{x}}_{\max} - \tilde{f} - \tilde{g} \end{bmatrix} \in \mathbb{R}^{4N_u+6N}. \tag{3.29}$$

3.3 Dynamic Controller

To implement the MPC at the dynamic level (2.6), it is necessary to compute the optimal control solution τ^* and execute the receding optimization. Receding optimization refers to the method employed in MPC where the optimization problem is solved repeatedly at each time step for a moving (or "receding") prediction horizon. Specifically:

1. At each time step k, the controller computes an optimal control sequence $u(k), u(k+1), \ldots, u(k+N)$ over a finite prediction horizon N.
2. Only the first control input $u(k)$ from the optimized sequence is applied to the system.
3. At the next time step $k+1$, the process is repeated by shifting the prediction horizon forward, using the updated state $x(k+1)$ as the starting point.

This "receding" strategy ensures that the optimization adapts to real-time changes in the system state and environment while keeping the computation manageable (finite horizon) instead of solving for an infinite horizon.

A torque control law $\tau = [\tau_1, \tau_2]^T$ is formulated to stabilize the system (2.6). τ_1 controls the force driving the robot's linear motion and τ_2 controls the torque affecting the robot's rotational motion. In addition to these control inputs, external disturbances and forces such as friction, drag, or unknown dynamics (e.g., uneven terrain) also act on the system. These disturbances are denoted as:

- $\bar{F}_1$ and $\bar{F}_2$: External forces in the linear and angular directions, respectively.
- $\bar{d}_1$ and $\bar{d}_2$: Unmodeled dynamics or friction affecting linear and angular motion.

The net force and torque are then given by:

$$F_{\text{net}} = \tau_1 - \bar{F}_1 - \bar{d}_1 \tag{3.30}$$

$$\tau_{\text{net}} = \tau_2 - \bar{F}_2 - \bar{d}_2 \tag{3.31}$$

Using Newton's second law for linear motion:

$$F_{\text{net}} = m\dot{v}, \tag{3.32}$$

we substitute $F_{\text{net}} = \tau_1 - \bar{F}_1 - \bar{d}_1$, giving:

$$m\dot{v} = \tau_1 - \bar{F}_1 - \bar{d}_1 \tag{3.33}$$

Rearranging for $\dot{v}$:

$$\dot{v} = \frac{\tau_1 - \bar{F}_1 - \bar{d}_1}{m} \tag{3.34}$$

Using Newton's second law for rotational motion:

$$\tau_{\text{net}} = I\dot{\omega}, \tag{3.35}$$

we substitute $\tau_{\text{net}} = \tau_2 - \bar{F}_2 - \bar{d}_2$, giving:

$$I\dot{\omega} = \tau_2 - \bar{F}_2 - \bar{d}_2 \tag{3.36}$$

Rearranging for $\dot{\omega}$:

$$\dot{\omega} = \frac{\tau_2 - \bar{F}_2 - \bar{d}_2}{I} \tag{3.37}$$

The system (2.6) can therefore be rewritten as

$$\begin{aligned} \dot{v} &= \frac{\tau_1 - \bar{F}_1 - \bar{d}_1}{m} \\ \dot{\omega} &= \frac{\tau_2 - \bar{F}_2 - \bar{d}_2}{I} \end{aligned} \tag{3.38}$$

Let $\epsilon_1 = -\frac{\vec{F}_1 + \bar{\tau}_{d1}}{m}$ and $\epsilon_2 = -\frac{\vec{F}_2 + \bar{\tau}_{d2}}{I}$, then (3.38) becomes

$$\begin{aligned} \dot{v} &= \frac{1}{m}\tau_1 + \epsilon_1 \\ \dot{\omega} &= \frac{1}{I}\tau_2 + \epsilon_2 \end{aligned} \tag{3.39}$$

The discrete-time dynamic model can be obtained by applying (3.6) into (3.39)

$$\begin{aligned} \nu(k+1) &= \nu(k) + \frac{T}{m}\tau_1 + T\epsilon_1 \\ \omega(k+1) &= \omega(k) + \frac{T}{I}\tau_2 + T\epsilon_2 \end{aligned} \tag{3.40}$$

Let $\tau = [\tau_1, \tau_2]$ and $\epsilon = [\epsilon_1, \epsilon_2]$, then (3.40) can be rewritten as

$$u(k+1) = h(u(k), \tau(k), \epsilon(k)) \tag{3.41}$$

where

$$h(u(k), \tau(k), \epsilon(k)) = u(k) + \text{diag}\left[\frac{T}{m}, \frac{T}{I}\right]\tau(k) + T\epsilon(k) \tag{3.42}$$

$u(k)$ represents the control input vector at the discrete time step k, and it consists of the robot's linear velocity(v) and angular velocity(ω). The function diag represents the diagonal matrix operator in linear algebra. Combining (3.12) and (3.41) gives:

$$\mathbf{x}(k+2) = l(\mathbf{x}(k), u(k), \tau(k), \epsilon(k)) \tag{3.43}$$

where

$$l(\mathbf{x}(k), u(k), \tau(k), \epsilon(k)) = \mathbf{x}(k) + g(\mathbf{x}(k))u(k) + g(\mathbf{x}(k)) + g(\mathbf{x}(k))u(k)h(u(k), \tau(k), \epsilon(k)) \tag{3.44}$$

Considering the constraints of (3.43) as specified

$$\begin{aligned} \tau_{\min} \leq \tau(k) \leq \tau_{\max} \\ u_{\min} \leq u(k) \leq u_{\max} \\ \epsilon_{\min} \leq \epsilon(k) \leq \epsilon_{\max} \end{aligned} \tag{3.45}$$

where $\tau_{\min}$ and $\tau_{\max}$ are the lower and upper bounds of the actuator inputs, $u_{\min}$ and $u_{\max}$

are the lower and upper bounds of velocities, and $\epsilon_{\min}$ and $\epsilon_{\max}$ are the lower and upper bounds of external disturbances. Based on (3.43), the predictive model can be formulated as follows:

$$\mathbf{x}(k+j \mid k) = l(\mathbf{x}(k+j-2 \mid k), u(k+j-2 \mid j), \tau(k+j-2 \mid j), \epsilon(k+j-2 \mid k)) \quad (3.46)$$

where $j \in [1, \tilde{N}]$.

The quadratic objective function of the velocity and control torque with a predictive horizon $\bar{N}$ and a control horizon $\bar{N}_u$ can be given by

$$\varphi(k) = \sum_{j=1}^{\bar{N}} \mathbf{x}^T(k+j|k)\bar{Q}\mathbf{x}(k+j|k) + \sum_{j=1}^{\bar{N}_u} \tau^T(k+j-2|k)\bar{R}\tau(k+j-2|k) \quad (3.47)$$

where $\bar{Q}$ and $\bar{R}$ denote the symmetric positive definite weighting matrices.

The first part of the quadratic objective function is focused on the state of the robot(denoted as $x(k+j \mid k)$) at time step $k+j$ given the information at time k.

$$\sum_{j=1}^{N_u} \mathbf{x}^T(k+j|k)\bar{\mathbf{Q}}\mathbf{x}(k+j|k)$$

- **State Deviation:** The term $\mathbf{x}^T(k+j|k)\bar{Q}\mathbf{x}(k+j|k)$ represents the cost associated with the robot's state deviation from its desired trajectory at each time step within the prediction horizon. The objective is to minimize how much the robot deviates from the desired trajectory.
- **Weighting Matrix:** $\bar{\mathbf{Q}}$: This matrix defines how important each state variable (position, velocity, or orientation) is in the overall objective. Essentially, $\bar{\mathbf{Q}}$ allows us to prioritize certain aspects of the state over others based on our performance goals (e.g., more weight on orientation if path-following precision is more critical than velocity).

The second part of the objective function focuses on the **control input**, which in this system is the torque τ applied to the robot's wheels or actuators:

$$\sum_{j=1}^{N_u} \left[\tau^T(k+j-2|k)\bar{R}\,\tau(k+j-2|k)\right]$$

- **Control Input (Torque):** The term

$$\tau^T(k+j-2|k)\bar{R}\,\tau(k+j-2|k)$$

  represents the cost associated with the torque used to control the robot at each step within the prediction horizon. The control inputs τ need to be minimized to avoid excessive energy usage, wear on actuators, and unnecessary oscillations.

- **Weighting Matrix ($\bar{R}$):** $\bar{R}$ determines how much weight is given to the control effort. A larger value in $\bar{R}$ means that large torque values are penalized more. By minimizing the torque required to drive the system along the desired trajectory, the robot becomes more energy-efficient and smoother in its motion.

Therefore, the following dynamic predictive controller can be obtained:

$$\tau^* = \arg\min_{\tau} \varphi(k) \tag{3.48}$$

subject to

$$\begin{aligned}
\mathbf{x}(k+j|k) = l(&\mathbf{x}(k+j-2|k), u(k+j-2|k), \\
&\tau(k+j-2|k), \epsilon(k+j-2|k)) && (3.49)\\
&j \in [1, \bar{N}] && (3.50)\\
\tau_{\min} \le \tau(k+j|k) &\le \tau_{\max}, \quad j \in [0, \bar{N}-2] && (3.51)\\
\epsilon_{\min} \le \epsilon(k+j|k) &\le \epsilon_{\max}. && (3.52)
\end{aligned}$$

Chapter 4

Neural Dynamics-Based Online Optimization

To solve the quadratic programming (QP) problem for MPC efficiently and in real time, a primal-dual neural network (PDNN) has been proposed. The PDNN is designed using the framework of Linear Variational Inequalities (LVI) to overcome the computational challenges associated with online optimization.

LVI(Linear Variational Inequalities) is a mathematical framework used to address certain optimization problems and equilibrium conditions. LVI provides a way to model and solve problems where inequalities play a role, such as systems with constraints. Our control problem fits into the LVI framework because it involves a constrained optimization problem (QP) with both equality and inequality constraints. LVI unifies the handling of the primal variables (control inputs) and the dual variables (Lagrange multipliers) into a single framework. This reformulation allows the use of efficient, real-time solvers like the primal-dual neural network (PDNN) to solve the problem while ensuring stability, feasibility, and optimality.

Consider the constraints defined in equations (3.9)–(3.11). To facilitate the analysis, we define the following terms:

1. **Dual Decision Vector:**

   $y \in \mathbb{R}^{4N_u+6N}$, where y^- and y^+ represent its lower and upper bounds, respectively.

2. **Primal-Dual Decision Vector:**

   $$d \in \mathbb{R}^{6N_u+6N}, \quad \text{defined as:}$$

   $$d = \begin{bmatrix} \Delta\bar{u} \\ y \end{bmatrix}, \quad d^+ = \begin{bmatrix} \Delta\bar{u}_{\max} \\ +y^+ \end{bmatrix}, \quad d^- = \begin{bmatrix} \Delta\bar{u}_{\min} \\ -y^- \end{bmatrix} \in \mathbb{R}^{6N_u+6N} \tag{4.1}$$

   Here, u represents the decision vector, and $u_{\max}$, $u_{\min}$ represent its upper and lower bounds.

3. **Matrix Representation:** A combined matrix M and vector p

   $$M = \begin{bmatrix} W & -E^T \\ E & 0 \end{bmatrix} \in \mathbb{R}^{(6N_u+6N)\times(6N_u+6N)}$$

   $$p = \begin{bmatrix} c \\ -b \end{bmatrix} \in \mathbb{R}^{(6N_u+6N)} \tag{4.2}$$

The dual form refers to an alternative representation of an optimization problem that is derived from the original (or primal) problem. For any i, the element $y_i^+ \gg 0$ in y^+ is chosen as $+\infty$ in practice. Thus, the convex set $\mathbf{\Omega}$ can be presented as $\Omega = \{d \mid d^- \leq d \leq d^+\}$, where d is the primal-dual decision vector, the variables W and E are defined in (3.26) and (3.28).

Therefore, we are concerned with the following LVI: find $d^* \in \mathbf{\Omega} = \{d \mid d^- \leq d \leq d^+\}$ such that:

$$(Md^* + p)^T(d - d^*) \geq 0, \quad \forall d \in \mathbf{\Omega}. \tag{4.3}$$

The above LVI (53) is equivalent to the QP problem (31)–(33) [44]. To solve (53), the following PDNN model is considered:

$$\dot{d} = -d + P_\Omega(d - (Md + p)) \tag{54}$$

where $P_\Omega(\cdot)$ is the projection operator onto Ω and defined as

$$P_\Omega(d) = [P_\Omega(d_1), \ldots, P_\Omega(d_{6N_u+6N})]^T$$

with

$$P_\Omega(d_i) = \begin{cases} d_i^- & \text{if } d_i < d_i^- \\ d_i & \text{if } d_i^- \leq d_i \leq d_i^+ \\ d_i^+ & \text{if } d_i > d_i^+ \end{cases}, \quad \forall i \in \{1, \ldots, 6N_u + 6N\}.$$

If d is an equilibrium point of the neural network (54), then

$$d = P_{\mathbf{\Omega}}(d - (Md + p)) \tag{4.4}$$

Therefore, we can consider the following modified projection neural network:

$$\dot{d} = \gamma(I + M^T)\{P_{\mathbf{\Omega}}(d - (Md + p)) - d\} \tag{4.5}$$

where γ is a strictly-positive design parameter used to scale the convergence rate of the system. Then, the ith neuron form of the LVI-PDNN for (4.5) can be presented as

$$\dot{d}_i = \gamma \sum_{j=1}^{\varrho} \left\{ z_{ij} \left[P_{\mathbf{\Omega}}^{(j)} \sum_{k=1}^{\varrho} s_{ik} h_k - p_i \right] - d_j \right\} \tag{4.6}$$

where $i = 1, 2, \ldots, \varrho$, $\varrho = 5N_u + 20N$, z_{ij} represents the ij-th element of matrix $Z = I + M^T$, and s_{ik} represents the ik-th element of matrix $S = I - M$. The virtual circuit implementing the LVI-PDNN in (4.5) is composed of ϱ integrators, ϱ limiters, $2\varrho^2$ multipliers, and 5ϱ summers.

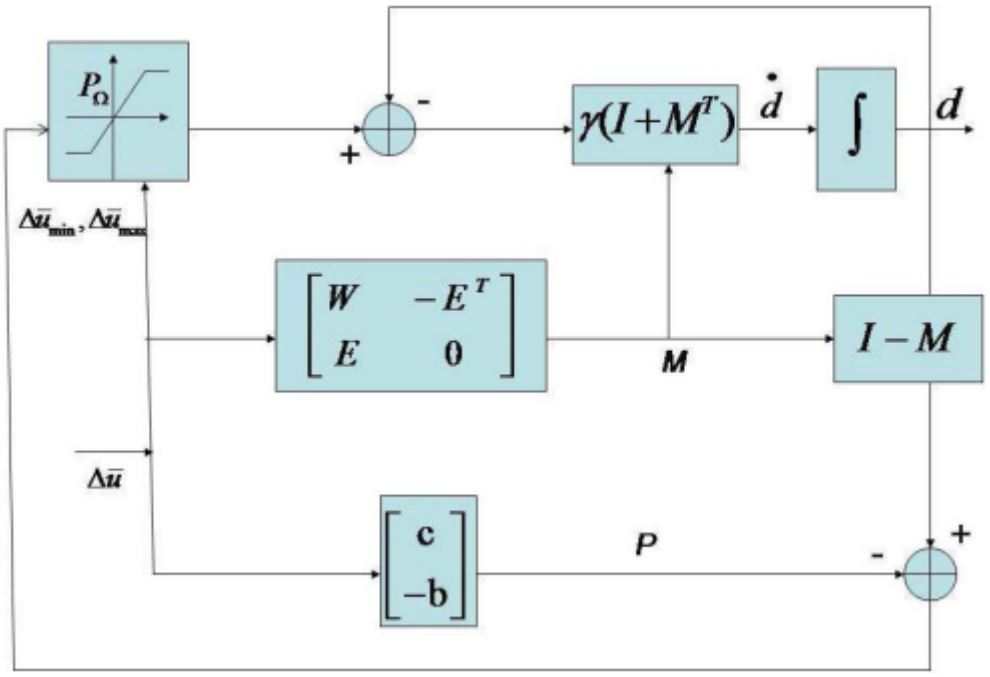


FIGURE 4.1: Block diagram of primal-dual neural dynamic system

Chapter 5

Stability Analysis

In this section, we will prove the stability of the developed MPC algorithm only at kinematics level. The stability and convergence analysis at the dynamics level can be performed in a similar manner and thus is omitted here. Based on the assumption that at least one optimal solution $\Delta\bar{u}^*$ of the QP problem formulated in (3.23)–(3.25) exist, it has been established in [**?**] that the state vector $\Delta\bar{u}$ of the LVI-based PDNN in (4.5) converges to equilibrium point $\Delta\bar{u}^*$ with arbitrary initial condition. Moreover, if there exists a constant $\rho > 0$ satisfying

$$\|d - P_\Omega(d - (Md + p))\|_2^2 \geq \rho\|d - d^*\|_2^2.$$

Then, the exponential convergence can be achieved for the LVI-based PDNN in (4.5) with convergence rate proportional to $\gamma\rho$. This inequality shows that the difference between the current and optimal state $(d - d^*)$ decreases exponentially with each iteration, ensuring faster convergence.

The QP's cost problem function is:

$$J(k) = \|\bar{\mathbf{x}}(k)\|_Q^2 + \|\Delta\bar{u}(k)\|_R^2$$

where:

- $\|\bar{x}(k)\|_Q^2$: Quadratic penalty on the deviation of the state vector $\bar{x}(k)$ from the desired state.
- $\|\bar{u}(k)\|_R^2$: Quadratic penalty on the control input vector $\bar{u}(k)$, discouraging excessive control effort.
- G: State transition matrix derived from system dynamics.
- f, g: Terms from the system model.

The expanded form of the function:

$$J(k) = \|G\Delta\bar{u}(k) + \bar{f} + \bar{g}\|_Q^2 + \|\Delta\bar{u}(k)\|_R^2 \tag{5.1}$$

This equation combines predicted system behavior ($G\bar{u}(k) + \bar{f} + \bar{g}$) and control efforts into a single metric to be minimized. Assuming that there exists at least one feasible solution for the QP problem, we can define the optimal value function as

$$V(\bar{\mathbf{x}}(k)) = \min \mathcal{J}(k). \tag{5.2}$$

The optimal value function is defined as the minimum cost achievable by the MPC and has the following key properties: The value function $V(\bar{x})$ has the following key properties:

1. **At the desired state ($\bar{x} = 0$):**

   $$V(0) = 0$$

   This means that when the system is at the desired state, there is no cost.

2. **For any non-zero state ($\bar{x} \neq 0$):**

   $$V(\bar{x}) > 0$$

   This implies a positive cost whenever there is deviation from the desired state.

3. **Continuity at $\bar{x} = 0$:**

   $$V(\bar{x}) \text{ is continuous at } \bar{x} = 0$$

This ensures smooth behavior near the desired state.

Let us denote a sequence of the difference of the optimal control input as

$$\Delta\bar{u}^*(k) = [\Delta u^*(k|k), \ldots, \Delta u^*(k+N_u-1|k)]^T \in \mathbb{R}^{N_u} \tag{5.3}$$

which satisfies the constraints (3.24)–(3.25), and denote the corresponding optimal state sequence as

$$\bar{\mathbf{x}}^*(k) = [\mathbf{x}^*(k+1|k), \ldots, \mathbf{x}^*(k+N|k)]^T \in \mathbb{R}^N \tag{5.4}$$

which satisfies the state constraints. Consider the fact that the optimal input sequence has minimized the optimization function $\mathcal{J}(k)$ such that

$$V(\bar{\mathbf{x}}^*(k)) = \|\bar{\mathbf{x}}^*(k)\|_Q^2 + \|\Delta\bar{u}^*(k)\|_R^2 \tag{5.5}$$

must be bounded.

If the prediction horizon N is chosen sufficiently large, then the equality of $x(k+i+1|k+1) = x^*(k+i+1|k)$, $1 \le i \le N$, would hold according to (3.49). Consequently, we have

$$\mathcal{J}(k+1) = \|\bar{x}(k+1)\|_Q^2 + \|\Delta\bar{u}(k+1)\|_R^2$$

$$= \|x(k+2|k+1)\|_Q^2 + \cdots + \|x(k+1+N|k+1)\|_Q^2 + \|\Delta u(k+1|k+1)\|_R^2 + \cdots + \|\Delta u(k+N_u|k+1)\|_R^2$$

$$\mathcal{J}(k+1) = V(\bar{\mathbf{x}}^*(k)) - \|\mathbf{x}^*(k|k)\|_Q^2 - \|\Delta u^*(k|k)\|_R^2. \tag{5.6}$$

Using the definition in (5.2), we can obtain

$$V(\bar{\mathbf{x}}^*(k+1)) \le \mathcal{J}(k+1)$$

$$V(\bar{\mathbf{x}}^*(k+1)) \le V(\bar{\mathbf{x}}^*(k)) - \|\mathbf{x}^*(k|k)\|_Q^2. \tag{5.7}$$

Obviously, we know that $V(\bar{\mathbf{x}}^*(k+1))$ is bounded. Consider the fact that the selected control input sequence (5.3) at time instant $k+1$ is also a feasible solution of the optimal question. Therefore, if the feasible solution for the optimal problem (3.18) exists at time

instant $k = 0$, then for arbitrary $k > 0$, the feasible solutions for the optimal problem also exist. As $\bar{\mathbf{x}}(k)$ approaches $\bar{\mathbf{x}}^*(k)$, we have the following inequality according to (5.7):

$$V(\bar{\mathbf{x}}(k+1)) - V(\bar{\mathbf{x}}(k)) \leq -\lambda_{\min}(Q)\|\bar{\mathbf{x}}^T(k)\|^2 \tag{5.8}$$

Then, we see that $V(\bar{\mathbf{x}}(k))$ converges along the trajectory of the system, such that the nominal stability of the closed-loop can be established.

Chapter 6

Experimental Results

6.1 Experimental Setup

The experimental platform was a TurtleBot 3 Waffle, configured for autonomous navigation tasks in a simulated environment using Gazebo. Gazebo provided a physics-based simulation of the TurtleBot within a virtual arena, allowing for precise control and monitoring of the robot's behavior under the tested conditions. The TurtleBot's dimensions were 281 mm x 306 mm x 141 mm, and it weighed approximately 1.8 kg. It had a maximum translational velocity of 0.26 m/s and a rotational velocity of 1.82 rad/s (104.27°/s), relying on Dynamixel XM430-W210-T actuators for motion.

In the simulated arena, the TurtleBot started at the world origin (0,0), navigating towards a 4x4 ArUco marker placed at coordinates (4.35, 1.5). The robot was equipped with an Intel RealSense camera to provide visual feedback essential for navigation.

The Intel RealSense camera simulated in Gazebo was configured to capture both RGB images and depth information. The RGB camera offered a resolution of 1920x1080 at 30 frames per second (fps), providing a detailed visual representation of the environment. The depth camera, with a resolution of 1280x720 at 30 fps, measured the distance to objects in the scene using infrared technology. Depth was calculated by measuring the time taken for infrared light to bounce back from surfaces. This allowed the camera to determine distances within a range of 0.2 m to 10 m. In this experiment, the depth readings were

critical for determining the TurtleBot's position relative to the ArUco marker and for providing the input to the control algorithms.

6.2 Control Implementation

6.2.1 MPC Controller

The Model Predictive Control (MPC) system was implemented using the CVXPY library to solve a quadratic programming optimization problem. The MPC operated with a sampling time of 0.1 seconds and a prediction horizon of 20 steps, making real-time control decisions at each time step. Two weight matrices governed the controller's performance: the state cost matrix Q and the control cost matrix R.

The state cost matrix Q, defined as:

$$Q = \text{diag}([2.0, 2.0, 5.0]),$$

penalized deviations from the target state. The values in Q show that the robot's x and y position errors were penalized equally (2.0), while greater emphasis (5.0) was placed on correcting orientation errors, which could have a larger impact on trajectory smoothness and path-following accuracy.

The control cost matrix R, defined as:

$$R = \text{diag}([0.01, 1]),$$

penalized the control inputs to avoid abrupt or excessive changes. The smaller value (0.01) for linear velocity changes indicates that the controller allowed for relatively more flexibility in adjusting speed, while the higher value (0.7) for angular velocity restricted sudden turns, promoting smoother rotations.

The controller solved a quadratic programming problem at each time step, minimizing the objective function while adhering to system constraints such as the robot's maximum allowable velocities of 0.5 m/s for linear motion and 0.5 rad/s for angular motion. By balancing the state and control penalties, the MPC ensured smooth, efficient navigation.

6.2.2 PD Controller

For comparison, a Proportional-Derivative (PD) controller was implemented, relying on feedback from the RealSense camera to adjust the robot's velocities. The PD controller's gains were set as follows:

$$K_p = \text{Linear: } 0.5, \text{Angular: } 0.001$$

$$K_d = \text{Linear: } 1e-10, \text{Angular: } 0.001$$

The proportional gain K_p for linear velocity ensured moderate correction based on positional error, while the angular proportional gain ensured gradual angular adjustments. The derivative gain K_d was used to dampen the controller's response, preventing oscillations, particularly in the angular domain.

6.2.3 Primal Dual Neural Network

The Primal-Dual Neural Network (PDNN) was initially considered for solving the quadratic programming (QP) problem within the Model Predictive Control (MPC) framework. The PDNN approach promised significant computational efficiency by leveraging neurodynamic optimization to solve QP problems in real time. This method would have replaced the traditional QP solver in the MPC, enabling faster and more parallelized computation of optimal control inputs.

The Primal-Dual Neural Network (PDNN) was designed to solve optimization problems of the form:

$$\min_{u} \ \frac{1}{2} u^T P u + q^T u$$

Subject to: - Inequality constraints:

$$Gu \leq h$$

- Equality constraints:

$$Au = b$$

where P, q, G, h, A, b are matrices and vectors derived from the Model Predictive Control (MPC) formulation.

Despite its potential, the implementation of the PDNN faced several challenges that prevented its successful integration into the control system. These challenges are summarized below.

Challenges in Implementing the PDNN

- Issues with Training Data: The training data required for supervised learning of the PDNN was generated using a traditional QP solver. However, inconsistencies and inaccuracies in this data led to suboptimal training performance. The dataset failed to capture sufficient diversity in operating conditions, such as varying robot states and constraints, which limited the generalization capabilities of the trained network.

- Implementation Complexity: The dynamic nature of the QP problem in MPC required frequent updates to matrices (P, q, G, h) at each time step. Incorporating these updates into the PDNN architecture introduced significant implementation complexity. Ensuring convergence of the primal-dual variables in real time while maintaining stability posed additional challenges.

- Lack of Resources and References: Limited availability of implementation resources and examples for applying PDNNs to QP problems in robotics hindered progress. Most existing research on PDNNs focused on theoretical analysis rather than practical implementation, leaving gaps in guidance for real-world applications.

6.3 Experimental Scenario

The experimental task involved visual servoing to guide the TurtleBot from the origin (0,0) to a stationary ArUco marker at coordinates (4.35, 1.5). The goal was for the TurtleBot to stop 0.5 meters from the marker. Both controllers—MPC and PD—were tested under this scenario to evaluate their effectiveness in navigating towards the visual target. The simulation setup in Gazebo allowed for testing the robot's behavior in a controlled, repeatable environment, simulating realistic sensor data.

6.4 Performance Evaluation

The performance of the MPC and PD controllers was evaluated based on time to reach the target, proximity to the target, and the smoothness of the path taken by the robot. The MPC controller reached the target in 8.88 seconds, while the PD controller required 9.52 seconds.

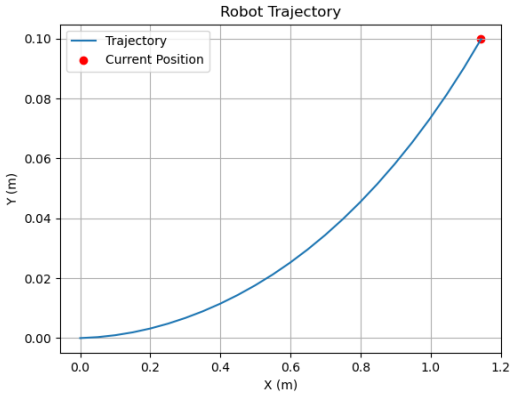


FIGURE 6.1: MPC Controller Trajectory

Despite the small time difference, the proximity results revealed a significant contrast. The MPC controller consistently achieved a final distance of exactly 0.5 meters from the marker, while the PD controller, due to minor oscillations, stopped at a slightly greater distance of 0.54 meters.

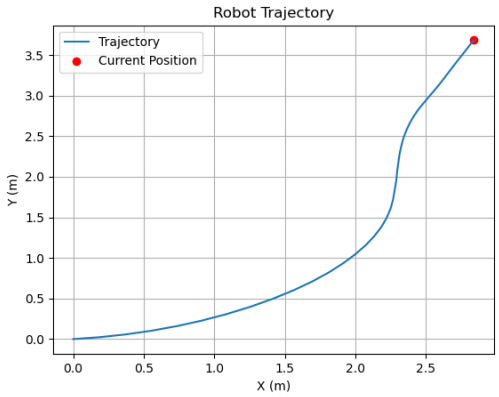


FIGURE 6.2: PD Controller Trajectory

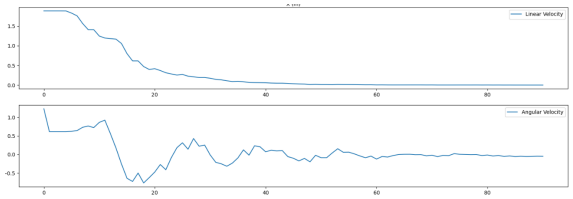


FIGURE 6.3: PD Controller Velocities

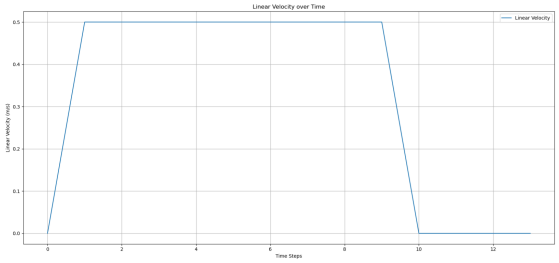


FIGURE 6.4: MPC Controller Linear Velocity

The path smoothness was another key metric. The MPC controller exhibited a smooth and stable trajectory, largely due to its ability to predict and optimize future states, allowing it to make fine adjustments to both linear and angular velocities. In contrast, the PD controller, which reacted directly to positional errors, produced a more erratic and jerky path, especially in its angular velocity.

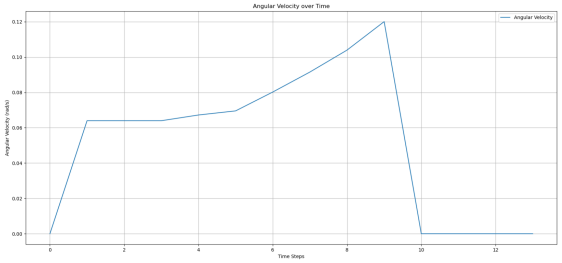


FIGURE 6.5: MPC Controller Angular Velocity

6.5 Visualization

Both controllers were visualized in real time. The robot's trajectory was plotted on a 2D plane, showing its path from the starting point to the target. The linear and angular velocities were also plotted over time, providing a clear picture of how each controller adjusted the robot's movements. The depth camera feed displayed the detected ArUco marker and calculated the distance to it, while time stamps overlaid on the visual feed tracked the robot's progress and total elapsed time.

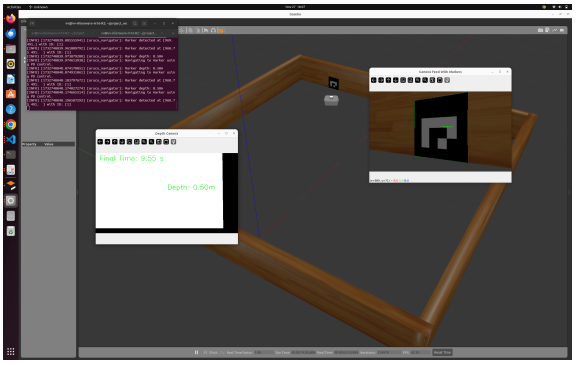


FIGURE 6.6: PD Controller Gazebo Visualization

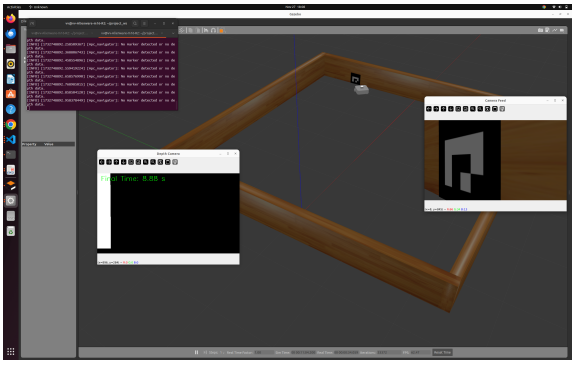


FIGURE 6.7: MPC Controller Gazebo Visualization

6.6 Optimization Framework

The MPC controller's optimization problem was formulated as a Quadratic Program (QP) and solved in real time using the OSQP solver via the CVXPY interface. The objective of the QP was to minimize the cost function:

$$\min_{\Delta\bar{u}} \quad \frac{1}{2}\Delta\bar{u}^T W \Delta\bar{u} + c^T \Delta\bar{u},$$

where W was a weighted sum of the state and control cost matrices Q and R, and c was derived from the state error. This formulation allowed the MPC to generate optimal control inputs while respecting constraints such as maximum velocity. By minimizing deviations from the target and avoiding abrupt changes in control inputs, the MPC maintained smooth, stable navigation, achieving the desired behavior in real time.

Chapter 7

Conclusion

The experimental results demonstrated the effectiveness of the implemented control strategies in achieving accurate and smooth navigation for the TurtleBot 3 Waffle in a simulated environment. The Model Predictive Control (MPC) controller generated better results as compared to the Proportional-Derivative (PD) controller in terms of trajectory smoothness, target proximity, and overall performance metrics.

The MPC controller successfully guided the robot to the target position with a final distance of exactly 0.5 meters, showcasing its ability to optimize control inputs while respecting system constraints. The smooth trajectory generated by MPC was attributed to its predictive nature, which allowed it to anticipate future states and make fine adjustments to both linear and angular velocities. In contrast, the PD controller exhibited minor oscillations and a slightly jerky path, stopping at a final distance of 0.54 meters from the target.

Comparison of MPC and PD Controllers

- Trajectory Smoothness: The MPC controller exhibited a smooth and stable trajectory, attributed to its ability to predict future states and optimize control inputs over a prediction horizon. This predictive capability allowed the robot to make fine adjustments to both linear and angular velocities, ensuring a seamless path to the target. In contrast, the PD controller produced a more erratic

trajectory, particularly in angular velocity. Its reactive nature caused minor oscillations and jerky movements as it adjusted control inputs based solely on instantaneous positional errors.

- Target Proximity: The MPC controller consistently achieved a final distance of exactly 0.5 meters from the target marker, demonstrating its precision in adhering to constraints and minimizing state errors. The PD controller stopped at a slightly greater distance of 0.54 meters due to its less refined control strategy, which struggled with precise adjustments near the target.

- Time to Reach Target: The MPC controller reached the target in 8.9 seconds, slightly faster than the PD controller, which required 9.45 seconds. This time difference can be attributed to MPC's proactive optimization of control inputs, enabling more efficient navigation.

- Handling Constraints: The MPC controller effectively handled system constraints such as maximum allowable velocities for linear (0.5 m/s) and angular motion (0.5 rad/s). By incorporating these constraints into its optimization problem, MPC ensured safe and reliable operation. The PD controller lacked explicit constraint handling capabilities, relying instead on manual tuning of gains to avoid exceeding limits.

Despite its theoretical promise, the Primal-Dual Neural Network (PDNN) could not be integrated into the control system due to several challenges. These included issues with training data quality, implementation complexity, and limited resources for practical guidance. The training data generated using traditional QP solvers lacked sufficient diversity and consistency, leading to suboptimal network performance. Additionally, frequent updates to problem matrices during real-time operation introduced significant implementation complexities. The lack of available references for applying PDNNs in robotic control further compounded these challenges.

Overall, the MPC controller proved to be a robust and reliable solution for visual servo-based navigation tasks, achieving real-time performance while maintaining smooth and stable control. Future work could focus on addressing the limitations encountered with

PDNN implementation by improving training data generation processes and exploring advanced computational resources for real-time optimization.

Bibliography

[1] C.-Y. Tsai and K.-T. Song, "Visual tracking control of a mobile robot using a new model in image plane," in *Proceedings of the IEEE 12th International Conference on Advanced Robotics (ICAR)*, July 2005, pp. 540–545.

[2] Y. Fang, X. Liu, and X. Zhang, "Adaptive active visual servoing of nonholonomic mobile robots," *IEEE Transactions on Industrial Electronics*, vol. 59, no. 1, pp. 486–497, Jan. 2012.

[3] K. Wang, Y. Liu, and L. Li, "Vision-based tracking control of nonholonomic mobile robots without position measurement," in *Proceedings of IEEE International Conference on Robotics and Automation*, May 2013, pp. 5265–5270.

[4] X. Zhang, Y. Fang, and X. Liu, "Motion-estimation-based visual servoing of nonholonomic mobile robots," *IEEE Transactions on Robotics*, vol. 27, no. 6, pp. 1167–1175, Dec. 2011.

[5] Z.-Y. Liang and C.-L. Wang, "Robust stabilization of nonholonomic chained form systems with uncertainties," *Acta Automatica Sinica*, vol. 37, no. 2, pp. 129–142, 2011.

[6] F. Yang and C.-L. Wang, "Adaptive stabilization for uncertain nonholonomic dynamic mobile robots based on visual servoing feedback," *Acta Automatica Sinica*, vol. 37, no. 7, pp. 857–864, 2011.

[7] Z. L. Wang and Y. H. Liu, "Visual regulation of a nonholonomic wheeled mobile robot with two points using lyapunov functions," in *Proceedings of IEEE International Conference on Mechatronics and Automation*, Aug. 2010, pp. 1603–1608.

[8] D. Gu and H. Hu, "Receding horizon tracking control of wheeled mobile robots," *IEEE Transactions on Control Systems Technology*, vol. 14, no. 4, pp. 743–749, Jul. 2006.

[9] G. Klančar and I. Škrjanc, "Tracking-error model-based predictive control for mobile robots in real time," *Robotics and Autonomous Systems*, vol. 55, no. 6, pp. 460–469, 2007.

[10] Y. Gao, C. G. Lee, and K. T. Chong, "Receding horizon tracking control for wheeled mobile robots with time-delay," *Journal of Mechanical Science and Technology*, vol. 22, no. 12, pp. 2403–2416, 2008.

[11] T. P. Nascimentoa, A. P. Moreira, and A. G. S. Conceicao, "Multi-robot nonlinear model predictive formation control: Moving target and target absence," *Robotics and Autonomous Systems*, vol. 61, no. 12, pp. 1502–1515, 2013.

[12] C. J. Ostafew, A. P. Schoellig, and T. D. Barfoot, "Learning-based nonlinear model predictive control to improve vision-based mobile robot path-tracking in challenging outdoor environments," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2014, pp. 4029–4036.

[13] H. Roggeman, J. Marzat, M. Sanfourche, and A. Plyer, "Embedded vision-based localization and model predictive control for autonomous exploration," in *Proc. IROS Workshop Vis. Control Mobile Robots*, Chicago, IL, USA, 2014, pp. 13–20.

[14] A. Aswani, H. Gonzalez, S. S. Sastry, and C. Tomlin, "Provably safe and robust learning-based model predictive control," *Automatica*, vol. 49, no. 5, pp. 1216–1226, 2013.

[15] T. M. Howard, C. J. Green, and A. Kelly, "Receding horizon model-predictive control for mobile robot navigation of intricate paths," in *Field and Service Robotics*. Berlin, Germany: Springer-Verlag, 2009, pp. 69–78.

[16] Y. Zhang, S. S. Ge, and T. H. Lee, "A unified quadratic-programming-based dynamical system approach to joint torque optimization of physically constrained redundant manipulators," *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, vol. 34, no. 5, pp. 2126–2132, Oct. 2004.

[17] Z. Li, S. S. Ge, and S. Liu, "Contact-force distribution optimization and control for quadruped robots using both gradient and adaptive neural networks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 25, no. 8, pp. 1460–1473, Aug. 2014.

[18] Z.-G. Hou, L. Cheng, and M. Tan, "Multicriteria optimization for coordination of redundant robots using a dual neural network," *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, vol. 40, no. 4, pp. 1075–1087, Aug. 2010.

[19] L.-X. Wang and F. Wan, "Structured neural networks for constrained model predictive control," *Automatica*, vol. 37, no. 8, pp. 1235–1243, 2001.

[20] M. Jiang, Y. Zhao, and Y. Shen, "A modified projection neural network for linear variational inequalities and quadratic optimization problems," in *Advances in Neural Networks (Lecture Notes in Computer Science)*. Berlin, Germany: Springer-Verlag, 2009, vol. 5553, pp. 1–9.

[21] A. Fusiello, E. Trucco, and A. Verri, "A compact algorithm for rectification of stereo pairs," *Machine Vision and Applications*, vol. 12, no. 1, pp. 16–22, 2000.

[22] R. W. Brockett, "Asymptotic stability and feedback stabilization," in *Differential Geometric Control Theory*, R. W. Brockett, R. S. Millman, and H. J. Sussmann, Eds. Boston, MA, USA: Birkhäuser, 1983, pp. 181–191.

[23] L. Grüne and J. Pannek, *Nonlinear Model Predictive Control: Theory and Algorithms*. London, U.K.: Springer-Verlag, 2011.

[24] J. Garriga and M. Soroush, "Model predictive control tuning methods: A review," *Industrial Engineering Chemistry Research*, vol. 49, no. 8, pp. 3505–3515, 2010.

[25] C.-T. Chen, *Linear System Theory and Design*. London, U.K.: Oxford University Press, 1995.

[26] WHO, "Who report on blindness," 2019, [WHO Official Report on Blindness]. [Online]. Available: https://www.who.int/publications/i/item/9789241516570

[27] IAPB, "Iapb report on blindness," 2019, [IAPB Official Report on Blindness]. [Online]. Available: https://www.iapb.org/learn/resources/the-world-report-on-vision/

[28] L. Kaczmirek and K. Wolff, "Survey design for visually impaired and blind people," in *Survey Design for Visually Impaired and Blind People*, vol. 4554, 07 2007, pp. 374–381.

[29] "Survey of visually impaired people and mobility." [Online]. Available: https://www.acb.org/content/survey-visually-impaired-people-and-mobility

[30] L. Kaczmirek and K. G. Wolff, "Survey design for visually impaired and blind people," in *Universal Access in Human-Computer Interaction. Part I, Coping with Diversity.* Springer, 2007, pp. 374–381.

[31] M. Goel, J. Ross, and J. O. Wobbrock, "Assistive technology for the blind and visually impaired in india: A study of the market and users' needs," *ACM Transactions on Accessible Computing (TACCESS)*, vol. 9, no. 2, p. 7, 2017.